1. Ejercicio 1. Refactorización y Control de Complejidad Ciclomática

- El siguiente fragmento de código ha sido rechazado por el analizador estático. Guarda este código en un archivo llamado `matricula.js`:

> [warning/error] complexity: Function 'calcularMatricula' has a cyclomatic complexity of 8 (max allowed: 5).
>
> [warning] max-depth: Blocks are nested 4 levels deep (max allowed: 2).
>
> [warning] no-else-return: Unnecessary 'else' after 'return'.

matricula.js - código a refactorizar (antipatrón)

```
// ANTIPATRÓN: Alta complejidad ciclomática y ramas 'else' innecesarias
function calcularMatricula(datos) {
  let resultado;

  if (datos !== null && typeof datos === "object") {
    if (typeof datos.precioBase === "number" && datos.precioBase > 0) {
      if (["general", "desempleado", "socio"].includes(datos.tipoEstudiante)) {
        let descuentoPorcentaje = 0;

        if (datos.tipoEstudiante === "desempleado") {
          descuentoPorcentaje = 0.30;
        } else {
          if (datos.tipoEstudiante === "socio") {
            descuentoPorcentaje = 0.15;
          }
        }

        let total = datos.precioBase * (1 - descuentoPorcentaje);

        if (datos.codigoDescuento === "WEB2026") {
          total = total - 10;
        }

        if (total < 0) {
          total = 0;
        }

        resultado = { valido: true, total: total };
      } else {
        resultado = { valido: false, error: "Tipo de estudiante no reconocido." };
      }
    } else {
      resultado = { valido: false, error: "El precio base debe ser un número positivo." };
    }
  } else {
    resultado = { valido: false, error: "Parámetros de matrícula inválidos." };
  }

  return resultado;
}
```

- Reescribir la función `calcularMatricula(datos)` cumpliendo los cinco requisitos:

- **Cláusulas de guarda (guard clauses):** las tres validaciones de precondición van al principio de la función y devuelven con un `return` temprano (`datos` no válido,

precio base no numérico o no positivo y tipo de estudiante desconocido), de modo que cuando se llega al cálculo ya se sabe que la entrada es correcta.

- **Sin anidamiento piramidal (arrow anti-pattern):** ningún `if` está dentro de otro `if`. Las guardas son hermanas directas del cuerpo de la función y su `return` es la única instrucción del bloque, así que la indentación máxima dentro de la función es de un nivel.
- **Sin ramas `else` innecesarias:** al cerrar cada `if` con un `return` no queda sitio para un `else`. Las tres guardas usan solo `if`, y las ramas de error ya no se escriben como `else`.
- **Camino principal despejado (happy path):** el descuento por tipo de estudiante, el cupón `WEB2026`, el suelo a cero y el `return` final quedan al final del cuerpo principal, a nivel de la función y sin estar dentro de ningún condicional.
- **Comportamiento idéntico:** se devuelve exactamente los mismos objetos `{ valido: false, error: "..." }` y `{ valido: true, total: ... }` para las mismas entradas.

El porcentaje de descuento se resuelve con una constante `DESCUENTOS_ESTUDIANTE` que ya contiene las tres claves válidas, en lugar de los dos `if` con `else` anidados del original: `Object.keys()` sobre esa misma constante hace de lista de tipos admitidos, así que la validación y el cálculo usan una única fuente y no hace falta repetir la comprobación en dos sitios.

matricula.js - función refactorizada con cláusulas de guarda

```
// REFACTORIZADO: cláusulas de guarda, sin anidamiento piramidal ni 'else'
const DESCUENTOS_ESTUDIANTE = Object.freeze({
  general: 0,
  desempleado: 0.30,
  socio: 0.15,
});

function calcularMatricula(datos) {
  if (datos === null || typeof datos !== "object") {
    return { valido: false, error: "Parámetros de matrícula inválidos." };
  }

  if (typeof datos.precioBase !== "number" || !(datos.precioBase > 0)) {
    return { valido: false, error: "El precio base debe ser un número positivo." };
  }

  if (!Object.keys(DESCUENTOS_ESTUDIANTE).includes(datos.tipoEstudiante)) {
    return { valido: false, error: "Tipo de estudiante no reconocido." };
  }

  const descuentoPorcentaje = DESCUENTOS_ESTUDIANTE[datos.tipoEstudiante];
  const descuentoWeb = datos.codigoDescuento === "WEB2026" ? 10 : 0;

  // Camino principal despejado (happy path)
  let total = datos.precioBase * (1 - descuentoPorcentaje) - descuentoWeb;

  if (total < 0) {
    total = 0;
  }

  return { valido: true, total: total };
}
```

- Medidas del código refactorizado:

Medida	Antipatrón	Refactorizado	Límite permitido
Complejidad ciclomática (`1 + nº de if`)	1 + 7 = **8**	1 + 4 = **5**	5
Ramas `else`	4	**0**	0
`if` anidado dentro de otro condicional	6	**0**	—

- Añadir el banco de pruebas al final del archivo para verificar la equivalencia funcional:

matricula.js - banco de pruebas (al final del archivo)

```
// Batería de pruebas: Comprobación de equivalencia funcional
console.assert(
  calcularMatricula(null).valido === false &&
  calcularMatricula(null).error === "Parámetros de matrícula inválidos.",
  "Fallo en Test 1: Manejo de entrada null"
);

console.assert(
  calcularMatricula({ precioBase: -20, tipoEstudiante: "general" }).valido === false &&
  calcularMatricula({ precioBase: -20, tipoEstudiante: "general" }).error === "El precio base debe ser un número posi
  "Fallo en Test 2: Precio base negativo o cero"
);

console.assert(
  calcularMatricula({ precioBase: 100, tipoEstudiante: "extranjero" }).valido === false &&
  calcularMatricula({ precioBase: 100, tipoEstudiante: "extranjero" }).error === "Tipo de estudiante no reconocido.",
  "Fallo en Test 3: Tipo de estudiante desconocido"
);

console.assert(
  calcularMatricula({ precioBase: 100, tipoEstudiante: "desempleado", codigoDescuento: "WEB2026" }).total === 60,
  "Fallo en Test 4: Descuento desempleado (30%) + cupón (-10 €)"
);

console.assert(
  calcularMatricula({ precioBase: 100, tipoEstudiante: "socio" }).total === 85,
  "Fallo en Test 5: Descuento socio (15%) sin cupón"
);

console.assert(
  calcularMatricula({ precioBase: 5, tipoEstudiante: "general", codigoDescuento: "WEB2026" }).total === 0,
  "Fallo en Test 6: Suelo mínimo a cero cuando el cupón excede el precio"
);

console.log("¡Todas las pruebas unitarias se ejecutaron con éxito!");
```

La salida en la consola del navegador es:

Consola del navegador

```
¡Todas las pruebas unitarias se ejecutaron con éxito!
```

Además de las seis aserciones, se compararon las dos versiones con 24 entradas de todo tipo (`null`, `NaN`, `Infinity`, precios a cero, cupones en minúsculas) y el resultado es idéntico en todas.